

Rapport : Projet de jeu d'aventure en Java

I.A) Auteur.....	3
I.B) Thème.....	3
I.C) Résumé du scénario.....	3
I.D) Plan.....	4
I.E) Scénario détaillé.....	5
I.F) Détail des lieux, items, personnages.....	5
I.G) Situations gagnantes et perdantes.....	6
I.H) Éventuellement énigmes, mini-jeux, combats, etc.....	6
I.I) Commentaires (ce qui manque, reste à faire, ...)	6
II. Réponses aux exercices (à partir de l'exercice 7.5 inclus).....	6
Exercice 7.5 :	6
Exercice 7.6 :	6
Exercice 7.7 :	7
Exercice 7.8 :	7
Exercice 7.9 / 7.10:.....	7
Exercice 7.10.2 :	7
Exercice 7.11 :	7
Exercice 7.14 :	7
Exercice 7.15 :	8
Exercice 7.16 :	8
Exercice 7.18 :	8
Exercice 7.18.5 :	8
Exercice 7.18.6 :	8
Exercice 7.18.7 :	8
Exercice 7.18.8 :	9
Exercice 7.20 :	9
Exercice 7.21 :	9
Exercice 7.22 :	10
Exercice 7.22.1 :	10
Exercice 7.23 :	10
Exercice 7.26 :	10
Exercice 7.28.1 :	10
Exercice 7.29 :	11
Exercice 7.30 :	11
Exercice 7.31 :	11
Exercice 7.31.1 :	12
Exercice 7.32 :	12
Exercice 7.33 :	12
Exercice 7.34 :	12
Exercice 7.42 :	12

Exercice 7.42.2 :	13
Exercice 7.43 :	13
Exercice 7.44 :	13
Exercice 7.46 :	14
Exercice 7.46.1 :	15
Exercice 7.47.1 :	15
Exercice 7.48 :	15
Exercice 7.49 :	16
Finalisation du projet :	16
III. Mode d'emploi (si nécessaire, instructions d'installation ou pour démarrer le jeu).....	17
IV. Sources.....	17
Code :	17
Images :	17
V. Déclaration obligatoire anti-plagiat.....	17

I.A) Auteur

Maxime LE BLAN
Groupe E3 - J2

I.B) Thème

“Sur la planète Kamino, un cadet Clone doit réussir l'épreuve de la Citadelle en récupérant son drapeau”

I.C) Résumé du scénario

Époque : il y a longtemps, dans la galaxie de l'univers Star Wars

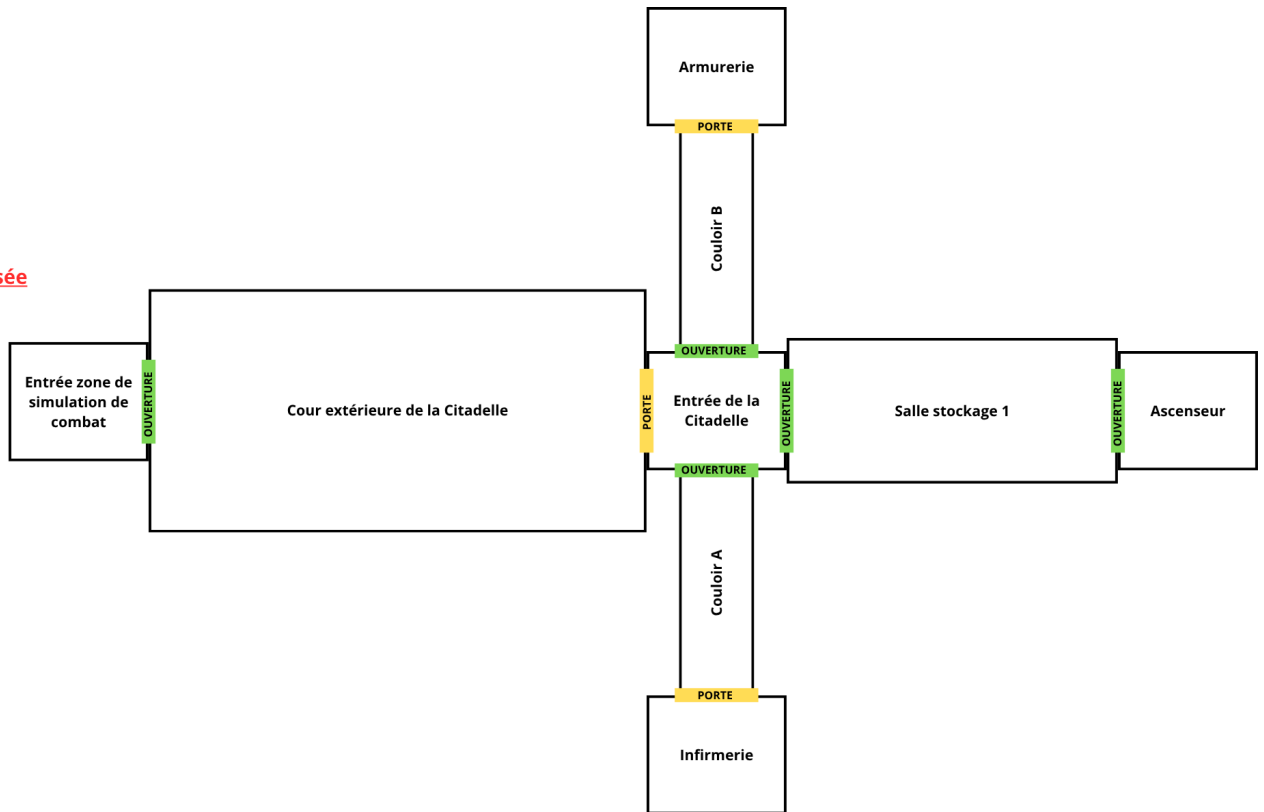
Lieu : à Tipoca City, sur la planète Kamino

Personnage : le cadet Clone CT-1904

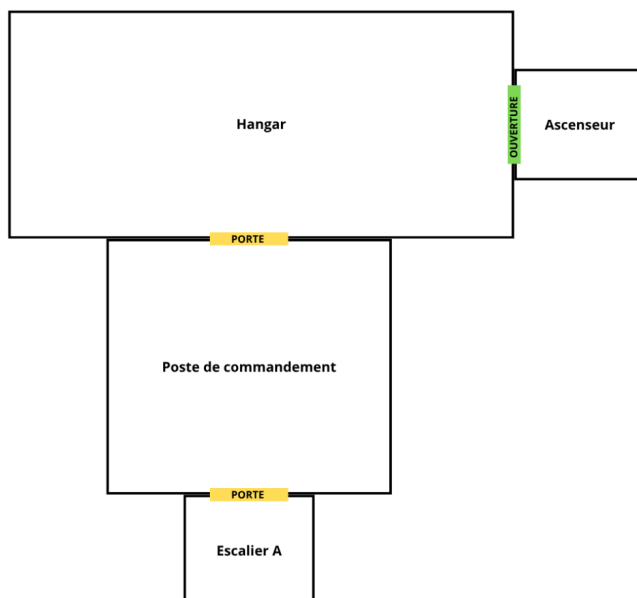
Objectif : Réussir l'épreuve de la Citadelle en récupérant son drapeau. L'épreuve de la Citadelle consiste à explorer le bâtiment principal de la Citadelle, et de ramasser le drapeau qui se situe sur le toit de celle-ci.

I.D) Plan

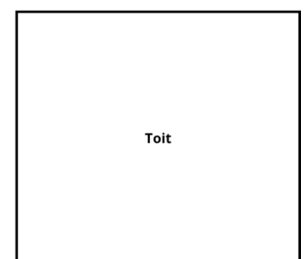
Rez de chaussée



Etage 1



Etage 2



I.E) Scénario détaillé

Le joueur débute la partie dans la pièce “Entrée zone de simulation” où le joueur pourra ramasser un couteau de combat et un fusil blaster DC-15S. Les autres items listés dans la partie précédente du rapport sont réparties dans différentes pièces du jeu. Concernant les PNJ, il y en a 3 : R2-D2 qui est un PNJ mouvant faisant des aller-retours entre l'armurerie et l'infirmierie en partant de la pièce “Entrée de la Citadelle”, puis R4-D7 et RA-7 qui sont des PNJ statiques présents dans la salle de stockage.

Le joueur peut se balader dans toutes les salles, sauf les pièces “Entrée zone simulation” et “Cour extérieure de la Citadelle” dans lesquelles il ne pourra plus rentrer une fois sorti de celles-ci (elles comportent toutes 2 des trapdoor). De plus, la pièce “Infirmierie” est une TransporterRoom, le joueur ne peut donc pas choisir dans quelle pièce il va atterrir en sortant de celle-ci.

Il peut également ramasser et déposer tous les objets qu'il souhaite dans toutes les pièces, et même poser des questions aux PNJ pour avoir plus d'informations sur certains d'entre eux. Cependant, les seuls objets utilisables sont le sac à dos, qui permet d'augmenter la capacité maximum d'objets que le joueur peut porter (équivalent du “magic cookie”), et la bague Dathomirienne qui se comporte comme un Beamer.

I.F) Détail des lieux, items, personnages

Liste des pièces :

- 1) entrée dans la zone de simulation au combat
- 2) cour extérieure de la Citadelle
- 3) entrée de la Citadelle
- 4) couloir A
- 5) couloir B
- 6) ascenseur (RC)
- 7) ascenseur (E1)
- 8) escalier
- 9) hangar
- 10) armurerie
- 11) infirmierie
- 12) poste de commandement de la Citadelle
- 13) salle de stockage
- 14) toit de la Citadelle

Liste des PNJ :

- R2-D2
- R4-D7
- RA-7

Liste des objets accessibles au joueurs :

- fusil blaster DC-15S
- fusil blaster DC-15A

- détonateur thermique
- détonateur ionique
- macro jumelles
- clé de crochetage
- médikit
- sac à dos de combat
- brouilleur de communication
- cartouches de Tibanna
- couteau de combat
- bague Dathomirienne
- drapeau

I.G) Situations gagnantes et perdantes

- Si le joueur effectue plus de 100 déplacements pendant la partie, alors il perd celle-ci et le jeu se termine.
- Une fois que le joueur accède au toit de la Citadelle et ramasse le drapeau, il remporte la partie.

I.H) Éventuellement énigmes, mini-jeux, combats, etc.

Aucun mini-jeu, énigme ou système de combat n'ont été implémentés. Les seules fonctionnalités présentes sont celles qui ont été implémentés entre les exercices 7.5 et 7.49.

I.I) Commentaires (ce qui manque, reste à faire, ...)

Le jeu est dans sa version finale.

II. Réponses aux exercices (à partir de l'exercice 7.5 inclus)

Exercice 7.5 :

La procédure `printLocationInfo` permet donc d'éviter la d'écrire 2 fois le même code dans les fonctions `goRoom` et `printWelcome`. Cela permet d'effectuer moins de modifications si on désire modifier le code contenu dans `printLocationInfo` (on aura uniquement une seule modification à effectuer au lieu de 2).

Exercice 7.6 :

1) Lorsque l'on déclare nos attributs directionnels de type `Room`, ne recevant par défaut aucune valeur lors de leur déclaration, ils possèdent donc la valeur `null`. Dans tous les cas, dans la fonction `setExits` si la valeur d'un paramètre `pDirectionExit` vaut `null`, qu'on range la valeur de `pDirectionExit` dans l'attribut correspondant ou non ne changera pas la valeur de celui-ci qui est dans tous les cas fixé à `null` par défaut.

Exercice 7.7 :

3) Etant donné que *Room* peut directement produire la liste des directions possibles et dans une logique de ne pas trop lier l'implémentation avec l'interface (et donc de ne pas laisser *Game* intervenir directement dans *Room*), il est plus logique de laisser *Room* produire la liste des directions possibles et d'autoriser *Game* à seulement récupérer ces informations pour ensuite les utiliser comme bon lui semble.

Exercice 7.8 :

La méthode *setExits* est désormais inutile car elle permettait de définir les sorties d'une pièce pour chaque direction. Or la méthode *setExit* permet d'associer une sortie à une direction, on peut donc utiliser plusieurs fois la méthode *setExit* à la place d'un *setExits*.

Par exemple :

```
vOutside.setExits(null, null, vLab, vTheatre);
```

est équivalent à :

```
vOutside.setExit("west", vLab);
```

```
vOutside.setExit("south", vTheatre);
```

Ce changement va nous permettre d'ajouter facilement des directions, car la méthode *SetExit* s'adapte à l'utilisation du *HashMap*.

Exercice 7.9 / 7.10:

L'instruction *tabAsso.keySet()* permet de renvoyer un tableau contenant toutes les clés du *HashMap* *tabAsso*. On peut donc ainsi modifier la méthode *getExitString* avec un *for* pour raccourcir le code et éviter de modifier la fonction lorsque l'on souhaite ajouter de nouvelles directions (telles que "up" et "down").

Exercice 7.10.2 :

La javadoc de la classe *Game* n'est pas visible car toutes les méthodes étant privées, leur javadoc associée ne s'affiche donc pas dans l'interface.

La méthode *getExitString()* a été implémentée dans la classe *Room* pour renvoyer toutes les informations concernant les sorties de la pièce qui sont accessibles.

Exercice 7.11 :

J'ai implémenté dans *Room* la méthode *getLongDescription()* qui renvoie toutes les informations concernant la pièce dans une chaîne de caractères, telles que la description de la pièce et ses sorties accessibles.

Exercice 7.14 :

J'ai implémenté dans *Game* la méthode *look(pCommande)* pour permettre au joueur d'afficher des informations sur la pièce dans laquelle il se trouve.

Exercice 7.15 :

J'ai implémenté dans *Game* la méthode *eat(pCommande)* pour permettre au joueur d'afficher un message indiquant qu'il vient de manger.

Exercice 7.16 :

J'ai implémenté dans la classe *CommandWords* la méthode *showAll()* pour afficher toutes les commandes valides du jeu, ainsi que la méthode *showCommands()* dans *Parser* qui permet d'afficher toutes les commandes du jeu avec la méthode précédente, en évitant d'augmenter le couplage entre les classes *Game* et *CommandWords*.

Exercice 7.18 :

J'ai d'abord modifié la méthode *showAll()* dans *CommandWords* en *getCommandList()* pour qu'elle renvoie la liste des commandes du jeu dans une chaîne de caractères au lieu de simplement l'afficher. J'ai donc également modifié *showCommands()* dans *Parser* pour de même simplement produire la liste des commandes du jeu au lieu de l'afficher.

Exercice 7.18.5 :

Pour accéder à toutes les pièces du jeu depuis n'importe quelle classe, j'ai pris l'initiative de créer dans la classe *Game* (puis dans *GameEngine* à partir du 7.18.6) la variable statique *HashMap<String, Room> sRooms*. Etant donné que notre projet ne contiendra qu'une seule instance de la classe *Game* (on ne programme qu'un seul jeu, pas plusieurs fois le même), il me semble plus adapté de déclarer la variable comme étant statique.

Exercice 7.18.6 :

J'ai effectué les modifications demandées dans l'énoncé de l'exercice en utilisant les nouvelles classes *GameEngine* et *UserInterface*.

Exercice 7.18.7 :

La commande *object.addActionListener()* permet de détecter une action effectuée sur *object*. Par exemple, si *object* est de type *JButton*, la méthode *addActionListener()* permet de détecter un appui sur *object*, et si *object* est de type *JTextField*, la méthode *addActionListener()* permet de détecter un appui sur la touche "Entrée" du clavier par défaut.

Lorsque *addActionListener()* est appelée, elle appelle alors à son tour automatiquement la méthode *actionPerformed(ActionEvent pE)* qui prend en

paramètre l'action qui a déclenché son appelle (par exemple un appui sur un bouton ou sur une touche du clavier), et `actionPerformed` exécute alors le code présent dans son corps de fonction.

Exercice 7.18.8 :

Pour implémenter un nouveau bouton *Help*, j'ai d'abord créé un nouvel attribut `private JButton aButton` après avoir importé la librairie `javax.swing.JButton`. Je l'ai ensuite positionné sur la partie gauche de l'encadré à l'aide de l'instruction `vPanel.add(this.aButton, BorderLayout.WEST)`. J'ai enfin pu connecter l'action correspondante au bouton avec l'instruction `this.aButton.addActionListener(this)` et modifier la fonction `actionPerformed` pour qu'elle appelle `interpretCommand` avec "help" comme paramètre afin d'afficher le message d'aide en cas d'appui sur le bouton par le joueur.

Exercice 7.20 :

Pour pouvoir implémenter des items dans mon jeu, j'ai donc commencé par créer la classe *Item* en lui ajoutant les attributs `aItemWeight` et `aItemDescription`, ainsi que les méthodes `getItemWeight()` et `getItemDescription()`.

Pour pouvoir les intégrer ensuite dans les pièces du jeu, j'ai créé dans *Room* l'attribut `aItem` pour pouvoir stocker un item. J'ai également ajouté les méthodes `setItem(pItem)` et `getItemString()` pour pouvoir ajouter un item dans chaque pièce si on le désire, et également informer au joueur quels items sont présents dans une pièce.

Enfin, j'ai ajouté la méthode `setItems()` (qui deviendra ensuite `createItems()`) dans *GameEngine* pour pouvoir créer et ajouter les items du jeu dans celui-ci.

Exercice 7.21 :

Pour garder une bonne cohésion entre les classes, il semble judicieux que les caractéristiques de chaque item soient données par la fonction `getItemLongDescription()` qui sera implémentée dans la classe *Item*.

Etant donné que nous avons précédemment affiché les informations des sorties de chaque *Room* en appelant la méthode de *Room* `getLongDescription()` dans la méthode `goRoom()` de la classe *GameEngine*, et que chaque *Room* contient au plus un item, il nous suffit de rajouter dans `getLongDescription()` le résultat de `getItemString()` (qui fait appel à `getItemLongDescription()`) en plus de ce qui était déjà présent. Ainsi à chaque appel de `goRoom()`, la description détaillée de l'item présent (ou bien "No item here" s'il n'y en a aucun) apparaîtra en complément des sorties existantes de la pièce.

Exercice 7.22 :

J'ai décidé d'utiliser un *HashMap* pour stocker les items de mon jeu dans chaque pièce, pour des raisons que je détaille dans l'exercice suivant. Mais avant d'implémenter le *HashMap* dans *Room*, j'ai d'abord ajouté l'attribut *aName* ainsi que son accesseur *getName()*. Enfin, j'ai changé dans *Room* l'ancien attribut *Item aItem* en *HashMap<String, Item> aItems* pour pouvoir stocker plusieurs items dans chaque pièce. J'ai ensuite remplacé la méthode *setItem(pItem)* par la méthode *addItem(pItem)* pour ajouter un item dans le *aItems*. J'ai dû également modifier la méthode *getItemString()* pour l'adapter au *HashMap*.

Exercice 7.22.1 :

Les collections qui sont à notre disposition en Java sont les *LinkedList* (listes chaînées), les *ArrayList* (tableaux) et les *HashMap* (ou *HashSet* qui est assez similaire). Dans notre cas, nous souhaitons accéder rapidement et clairement à chaque item de chaque pièce au sein de notre structure. Étant donné que **l'accès à un élément** dans une *LinkedList* de taille *n* **se fait en $O(n)$** , il n'est pas optimal d'utiliser cette structure de données. En ce qui concerne les *ArrayList*, nous n'avons pas ce problème car **l'accès se fait en $O(1)$** . En revanche, il faudrait associer à chaque item stocké dans le tableau son indice pour le retrouver facilement, ce qui n'est pas très pratique. La solution la plus simple est donc l'utilisation d'un *HashMap*, qui permet d'**accéder à un item en $O(1)$ facilement car chaque item est identifié dans le *HashMap* par son nom**.

Exercice 7.23 :

Avant d'implémenter la commande *back*, j'ai d'abord ajouté l'attribut *Room aPreviousRoom* dans *GameEngine* afin d'enregistrer la pièce où se trouvait précédemment le joueur. J'ai ensuite modifié la méthode *goRoom()* pour qu'elle modifie la valeur de l'attribut dès que l'on change de pièce. Enfin, j'ai ajouté la commande en écrivant la fonction *back()* dans *GameEngine* pour que le joueur puisse revenir dans la pièce précédente dès qu'il le souhaite.

Exercice 7.26 :

Au cours de cet exercice, j'ai d'abord changé l'attribut *Room aPreviousRoom* en *Stack<Room> aPreviousRooms*, puis j'ai uniquement eu besoin de modifier légèrement les méthodes *goRoom()* et *back()* pour pouvoir les adapter au changement de type de *aPreviousRoom*.

Exercice 7.28.1 :

Pour cet exercice, je suis d'abord allé voir dans la rubrique "Plus de technique" pour comprendre comment lire des fichiers en Java, puis j'ai repris le code de la classe

LectureFichierSimple que vous aviez écrit dans celle-ci. Je l'ai ensuite modifiée pour que la méthode `lecture(pNomFichier, pGameEngine)` puisse lire chaque commande écrite dans le fichier texte et appeler la méthode `interpretCommand(pCommandLine)` de *GameEngine* sur cette commande.

J'ai ensuite implémenté la méthode `test(pCommande)` dans *GameEngine* en utilisant la méthode `lecture` présente dans *LectureFichierSimple*.

Exercice 7.29 :

Pour cet exercice, j'ai d'abord pris le temps d'implémenter dans un premier temps la classe *Player*. Je lui ai donc ajouté les attributs `aCurrentRoom`, `aName` et `Stack<Room> aPreviousRooms`, ainsi que les méthodes `getCurrentRoom()`, `setCurrentRoom(pNewCurrentRoom)`, `addPreviousRoom(pRoom)` et `popPreviousRoom()`. On peut ici remarquer que les attributs `aCurrentRoom` et `aPreviousRooms` étaient auparavant dans *GameEngine* et ont été déplacés dans *Player*, ce qui est logique vu qu'ils stockent des informations liées au joueur.

Dans un second temps, j'ai dû modifier la plupart des méthodes implémentant les commandes du jeu dans *GameEngine*, et en particulier les méthodes `goRoom` et `back`. J'ai décidé de garder une méthode par commande dans *GameEngine*, mais contrairement à auparavant, *GameEngine* sert maintenant uniquement à gérer l'affichage et à appeler des méthodes d'autres classes, telles que *Player*, pour pouvoir effectuer des actions dans le jeu.

J'ai également ajouté une instance de *Player* dans le constructeur de *GameEngine* pour pouvoir ajouter le joueur dans le jeu.

Exercice 7.30 :

J'ai d'abord effectué des modifications dans la classe *Player* en ajoutant l'attribut `aItem` ainsi que les méthodes `take(pItem)`, `drop()` et également `hasItems()` (pas demandée dans l'exercice mais qui pourrait me servir plus tard). Ensuite, j'ai dû ajouter la méthode `removeItem(pItem)` dans *Room* pour pouvoir l'utiliser dans la fonction `drop()`.

Enfin, j'ai implémenté les méthodes `take(pCommande)` et `drop(pCommande)` dans *GameEngine* pour pouvoir gérer l'affichage.

Exercice 7.31 :

Dans cet exercice, j'ai uniquement eu besoin de changer dans *Player* l'attribut `altm` en `HashMap<String, Item> aInventaire` pour pouvoir stocker plusieurs items. Il a fallu ensuite que j'adapte les méthodes `take(pItem)` et `drop()` pour qu'elles puissent à nouveau fonctionner.

Exercice 7.31.1 :

J'ai décidé de choisir un *HashMap* comme structure de données pour la classe *ItemList* car cela permet d'accéder ou de modifier un élément de la *ItemList* uniquement avec le nom de l'item que l'on veut ajouter, modifier ou supprimer, ce qui est bien plus pratique ici qu'avec une *ArrayList* par exemple.

J'ai donc commencé par implémenter la classe *ItemList* en lui ajoutant l'attribut *HashMap<String, Item> aItemList*, ainsi que les méthodes *get(pItemName)*, *add(pItem)*, *remove(pItemName)* et *getItemString()*. Enfin, il ne me restait plus qu'à changer le type des attributs *aItems* dans *Room* et *aInventaire* dans *Player* en *ItemList*, puis à faire de légères modifications aux méthodes de ces deux classes qui utilisaient auparavant un *HashMap*.

Exercice 7.32 :

J'ai d'abord ajouté l'attribut *aMaxWeight* dans la classe *Player* avant de mettre sa valeur à 8 dans le constructeur de la classe. J'ai ensuite ajouté la méthode *canAddToInventory(pItem)* pour déterminer si un item peut être ajouté dans l'inventaire du joueur, et j'ai modifié la méthode *take* pour qu'elle effectue ce test avant d'ajouter ou non un item dans l'inventaire.

Exercice 7.33 :

Pour pouvoir ajouter la commande *items*, j'ai d'abord dû ajouter la méthode *getTotalWeight()* dans la classe *ItemList*, puis écrire la méthode *getInventoryWeight()* ainsi que *getInventoryString()* dans *Player*. Enfin, j'ai écrit la méthode *items(pCommande)* dans *GameEngine* qui utilise donc les méthodes écrites préalablement dans *Player*.

Exercice 7.34 :

Etant donné qu'un item "sac_à_dos" était déjà implémenté dans mon jeu d'aventure pour pouvoir augmenter le poids maximal d'items que le joueur peut porter, j'ai pris l'initiative de créer une commande supplémentaire *use* pour le joueur fonctionnant exactement comme la commande *eat* que l'on doit implémenter dans l'exercice avec le "magic cookie". J'ai uniquement effectué ce changement dans le but d'être cohérent avec le fait qu'on utilise un sac à dos, on ne le "mange" pas.

La fonction *eat* est donc toujours présente dans le jeu, mais fonctionne comme à l'exercice 7.15.

Exercice 7.42 :

J'ai décidé dans mon jeu de définir la limite de temps en comptant le nombre de déplacements total que le joueur a effectué depuis le début de la partie. Pour cela, j'ai défini un nouvel attribut *aMovingLimit* dans *GameEngine*, car c'est son rôle de déterminer si la partie est terminée ou non. Cependant, le nombre de pas qu'effectue le joueur ne devrait pas être géré par *GameEngine*, mais plutôt par *Player* si l'on veut rester cohérent avec les rôles de chaque classe. J'ai donc créé l'attribut *aMovingCounter* dans *Player* ainsi que son accesseur correspondant *getMovingNumber()*. J'ai également modifié dans *Player* la fonction *setCurrentRoom* pour qu'elle incrémente le compteur dès qu'elle est appelée.

Quant à *GameEngine*, j'ai créé la fonction *gameIsFinished()* qui permet de déterminer si la partie est terminée, et je l'appelle dans *interpretCommand*. De ce fait, à chaque fois que le joueur rentre une commande, on vérifie s'il peut encore continuer la partie ou non et sinon on arrête la partie. C'est ce que vérifie le bout de code suivant dans *interpretCommand* :

```
if (this.gameIsFinished())
{
    this.endGame();
    return;
}
```

Exercice 7.42.2 :

J'ai décidé de ne pas faire d'IHM graphique car je préfère me concentrer en priorité sur les fonctionnalités du jeu.

Exercice 7.43 :

Afin de créer mes "trapdoor", j'ai décidé d'adopter la méthode consistant à supprimer la sortie faisant office de trapdoor dans la pièce concernée. On peut donc juste supprimer une seule ligne dans le code lorsque l'on veut créer une trapdoor supplémentaire. Par exemple, comme je voulais que la sortie ouest de la pièce "Cour extérieure de la Citadelle" devienne une trapdoor, j'ai supprimé la ligne suivante :

```
vCourExt.setExit("west", vEntreeSim);
```

J'ai également ajouté la méthode *isExit(pRoom)* dans la classe *Room* et j'ai modifié la fonction *popPreviousRoom()* dans la classe *Player* pour gérer le problème de la commande *back*. Dans *popPreviousRoom()*, je vide la pile si *this.aCurrentRoom.isExit(this.aPreviousRooms.peek())* vaut *false* et je retourne la pièce courante.

Exercice 7.44 :

Afin d'intégrer le beamer dans le jeu, j'ai donc créé une classe *Beamer* qui hérite de la classe *Item*. J'y ai ajouté les méthodes *isLoading()*, *charge(pRoom)* et *fire()* afin de pouvoir utiliser le beamer.

Pour que le joueur puisse interagir avec celui-ci, j'ai dû modifier la fonction *useItem* dans *Player* qui servait déjà à utiliser le sac à dos (alias "magic cookie") pour qu'elle puisse inclure l'utilisation du beamer (je sous-entend l'utilisation du mode "fire" du beamer) et j'ai rajouté une méthode *charge* qui appelle la méthode *charge* du beamer.

Pour finir, j'ai écrit la méthode *charge* et modifié la méthode *use* dans la classe *GameEngine* afin de gérer l'affichage indiquant au joueur les actions qu'il a effectué. J'ai donc remplacé la commande *fire* que l'on nous demande d'implémenter par la commande *use*, qui servait déjà auparavant à gérer le sac à dos. Il me semblait logique dans notre cas de rassembler les deux opérations dans la même méthode étant donné que *use* gère déjà l'utilisation d'items, et que peu de code nécessitait d'être modifié. Mais dans le cas où l'on aurait eu un nombre bien plus important d'items ainsi qu'une plus grande diversité, il aurait peut-être été plus intéressant de séparer les actions de *use* dans *eat* et *fire*.

J'ai également ajouté une ligne dans *useItem* pour vider la pile contenant les pièces précédentes afin d'empêcher l'utilisation d'un *back* après utilisation du beamer.

Exercice 7.46 :

Comme suggéré dans le chapitre 9.11, j'ai créé la classe *RoomRandomizer* qui contient la fonction statique *findRandomRoom()* permettant de retourner une pièce sélectionnée aléatoirement parmi celles existantes dans le jeu. J'ai ensuite créé la classe *TransporterRoom* qui est une classe héritière de la classe *Room*, et qui contient uniquement une réécriture de la fonction *getExit(pDirection)* en renvoyant une pièce aléatoire différente de la "transporter room" avec *findRandomRoom()*, au lieu de la pièce associée à la sortie *pDirection* comme auparavant.

Pour ce qui est du fonctionnement de la fonction *findRandomRoom()*, étant donné que les pièces du jeu ont été stockées dans le HashMap *GameEngine.sRooms* et que la fonction *nextInt(bound)* de la classe *Random* ne renvoie qu'un nombre entier compris entre [0 ; bound [, j'ai pris l'initiative de créer un ArrayList pour recopier toutes les pièces stockées dans le HashMap, puis j'accède à l'une d'entre elles grâce à l'indice obtenu aléatoirement avec la fonction *nextInt(bound)*.

Pour finir, j'ai juste eu à changer le type déclaré de la pièce *vInfirmier* dans *GameEngine* en le passant de *Room* à *TransporterRoom* pour avoir une "transporter room" fonctionnelle dans le jeu.

Exercice 7.46.1 :

Avant d'implémenter la commande, j'ai ajouté à *GameEngine* l'attribut *aTestMod* qui contient un booléen indiquant si le jeu est en mode test ou non, et qui est mis à *false* par le constructeur. J'ai également rajouté une ligne dans la méthode test qui passe la valeur de *aTestMod* à *true* lorsqu'elle est exécutée par le joueur.

J'ai ensuite ajouté l'attribut *aNextRandomRoom* dans la classe *TransporterRoom* et j'ai créé son modificateur associé *setNextRandomRoom*, puis j'ai modifié la fonction *getExit* pour qu'elle puisse prendre en compte une éventuelle pièce où l'on veut aller une fois sorti de la transporter room.

Enfin, j'ai ajouté la commande *alea* dans *GameEngine* qui fonctionne comme expliqué dans l'énoncé de l'exercice.

Exercice 7.47.1 :

J'ai décidé dans le cadre de mon projet de créer 2 paquetages respectivement nommés *pkg_items* et *pkg_commands* qui se situent dans le paquetage courant. Le premier contient donc les classes *Command* et *CommandWords*, et le second contient les classes *Item* et *ItemList*. Ces classes étant utilisées dans de nombreuses autres classes et étant de bas niveau (elles importent peu de bibliothèques externes), il semble donc judicieux de les mettre dans des paquetages agencés de cette manière.

Il ne me restait donc ensuite qu'à ajouter les imports nécessaires dans les classes telles que *GameEngine*, *Room*, *Player* etc... qui utilisent ces 2 paquetages.

Exercice 7.48 :

Dans le but d'introduire des PNJ dans mon jeu, j'ai donc dû créer une classe *Character* chargée d'implémenter les fonctionnalités principales des mes PNJ. Dans mon jeu, une salle peut contenir un ou plusieurs PNJ, et chaque PNJ contient des informations sur certains items du jeu, que le joueur peut obtenir en tapant la commande "*ask <nom-du-PNJ> <nom-de-l'item>*". La première fois que le joueur pose une question à un PNJ, il se présente d'abord avec un cours texte qu'il ne répètera plus ensuite, puis il répond à la question du joueur.

Pour mettre en place ces fonctionnalités, j'ai d'abord dû écrire la classe *Character* qui contient les attributs *aName*, *aItemsInfo*, *aPlayersMeeting* et *aDefaultText*, ainsi que les méthodes *getName()*, *hasMetPlayer(pPlayerName)*, *meetPlayer(pPlayerName)*, *getIntroductionText()*, *addItemInfo(pItemName, pInformation)* et *getItemInfo(pItemName)*. Le fonctionnement et le rôle de toutes ces méthodes et tous ces attributs sont détaillés dans la javadoc.

J'ai ensuite dû rajouter en attribut de la classe *Room* le *HashMap<String, Character>* *aCharacters* pour contenir tous les PNJ présents dans une pièce avec comme couple (clé, valeur) le nom du PNJ avec le PNJ lui-même. A cela, j'ai également ajouté les méthodes *addCharacter(pNPC)*, *getCharacter(pNPCName)* et *getCharacterString()*. La fonction *getLongDescription()* a également été modifiée pour pouvoir afficher les PNJ présents dans chaque pièce en plus des items et des sorties.

J'ai ensuite modifié la classe *Command* pour pouvoir ajouter un troisième mot clé aux commandes tapées par le joueur, ainsi que la classe *Parser* pour le traitement des entrées. J'ai ajouté dans *Command* l'attribut *aThirdWord* ainsi que les méthodes *getThirdWord()* et *hasThirdWord()*.

Enfin, j'ai rajouté la commande *ask* décrite précédemment dans *GameEngine* pour que le joueur puisse poser des questions sur des items aux PNJ.

Exercice 7.49 :

Étant donné que les PNJ qui se déplacent sont une sorte de PNJ, j'ai décidé de les implémenter dans une nouvelle classe *MovingCharacter* qui hérite de la classe *Character*. Dans mon jeu, les PNJ pouvant se déplacer se comportent comme des PNJ classiques, mis à part qu'ils se déplacent lorsque le joueur tape la commande *go*, et ils suivent un chemin préconçu qu'ils effectuent en boucle.

Pour ce faire, j'ai donc écrit la classe *MovingCharacter*, qui comporte les attributs *aCurrentRoom*, *aPath* de type *ArrayList<Room>* et *aINextRoom* ainsi que les méthodes *move()*, *addToPath(pRoom)* et *getCurrentRoom()*. Le chemin que suit un PNJ mouvant est donc stocké dans le tableau *aPath*, et l'on peut le construire en ajoutant des pièces à l'aide de la fonction *addToPath*.

Pour pouvoir intégrer leurs déplacements dans le jeu, j'ai donc dû modifier les classes *Room* et *GameEngine* en ajoutant dans *Room* la méthode *removeCharacter(pNPCName)*, et en ajoutant dans *GameEngine* l'attribut *HashMap<String, MovingCharacter>* *aMovingCharacters* et en modifiant la méthode *goRoom(pCommande)*. Le *HashMap* sert à stocker tous les PNJ mouvant existants dans le jeu pour que l'on puisse leur ordonner de tous se déplacer dès que le joueur change de place avec des exécutions successives de la méthode *move()*.

Finalisation du projet :

Pour terminer, j'ai implémenté la fin du jeu dans le cas où le joueur gagne. J'ai ajouté l'item *drapeau* dans la pièce du toit de la Citadelle, et j'ai modifié la fonction *gameIsFinished()* dans *GameEngine* pour qu'elle regarde si le joueur possède cet item dans son inventaire, et si oui, alors le joueur a gagné la partie. J'ai ensuite ajouté un appel

supplémentaire à cette fonction à la fin de la fonction *take* pour effectuer la vérification après chaque ramassage d'item.

III. Mode d'emploi (si nécessaire, instructions d'installation ou pour démarrer le jeu)

Exécuter la fonction *test()* de la classe *Test* pour pouvoir lancer le jeu.

IV. Sources

Code :

- Certaines parties du code sont des copies directes du site ci-dessous :
<https://perso.esiee.fr/~bureaud/Unites/Zuul/listeExercices.htm>

Images :

- Captures d'écran du jeu vidéo Star Wars Battlefront 2 (2017) effectuées sur mon ordinateur.
- Images modifiées par Gemini ayant pour origine :
 - <https://sebastinpinzn.artstation.com/projects/v1nk6E>
 - <https://www.artstation.com/artwork/d8QEPW>

V. Déclaration obligatoire anti-plagiat

Je soussigné, Maxime LE BLAN :

- Certifie qu'il s'agit d'un travail original et que toutes les sources utilisées ont été indiquées dans leur totalité.
- Certifie enfin que ce mémoire, totalement ou partiellement, n'a jamais été évalué auparavant et n'a jamais été édité.

Fait à Champs-sur-Marne, le 14 / 01 / 2026